

Energy-Delay Efficient Segmented Approximate Adder With Smart Chaining

Tayebbeh Karimi  and Arezoo Kamran 

Abstract—Approximate computing is a promising approach for high-performance, and low-energy computation in inherently error-tolerant applications. This paper proposes an approximate adder comprising a constant-truncation block in the least significant part and several non-overlapping summation blocks in the more significant parts of the adder. The carry-in of each block is supplied using the most significant bit of one of the input operands from the earlier block. In the most significant block, two more-precise approaches are used to generate candidate values for the carry-in. The final value of the carry-in for this block is selected based on the values of the input operands. In fact, the proposed approximate adder is input-aware, and dynamically adjusts its operation in one or two cycles to improve accuracy while limiting the average delay. The experimental results indicate that the proposed adder has a better quality-effort tradeoff than state-of-the-art approximate adders. Different configurations of the proposed adder improve delay, energy, and the energy-delay product (EDP) by 78%, 72%, and 87%, respectively, when compared to state-of-the-art approximate adders, all without any loss in accuracy. Additionally, the efficiency of the proposed adder is confirmed in both image dithering and stock price prediction through regression.

Index Terms—Approximate adder, low-energy computation, approximate image processing, image dithering, stock price prediction, regression.

I. INTRODUCTION

HIGH-PERFORMANCE, low-power computing has always been challenging. The rise of artificial intelligence (AI), big data processing, and portable multimedia devices has increased the importance of efficient computing systems. Various design metrics, including latency, and power/energy often conflict with each other, where improving one can degrade the other. Traditional approaches to addressing this tradeoff in design metrics, such as low-power design techniques and many-core systems that minimize the complexity of processing cores and utilize parallel execution concepts, have approached the saturation points.

Received 17 March 2024; revised 9 October 2024; accepted 10 November 2024. Date of publication 18 November 2024; date of current version 20 January 2025. Recommended for acceptance by N. Revol. (Corresponding author: Arezoo Kamran.)

The authors are with the Razi University, Kermanshah 6714414971, Iran (e-mail: kamran@razi.ac.ir).

Digital Object Identifier 10.1109/TC.2024.3500371

Approximate computing (AC) is an emerging paradigm aimed at improving the efficiency-energy tradeoff by trading accuracy for energy and speed. Many applications do not require exact computing and, exhibit inherent resilience to erroneous results. This error resilience arises from limited human perception, the absence of a golden solution in some algorithms, noise in the input data, and errors in analog-to-digital conversion. AI, machine learning (ML), data mining, image processing, audio processing, wireless communication, and many other fields can benefit from AC.

AC techniques can be applied to various layers of the hardware-software stack, from circuits to applications [1], [2]. Approximate arithmetic circuits are a key focus of AC research, with a variety of architectures proposed [3], [4], [5], [6], [7], [8].

In this study, we propose an approximate adder called Segmented Approximate Adder with Smart Chaining (SASC). SASC has a better quality-effort tradeoff than state-of-the-art approximate adders. SASC comprises several non-overlapping summation blocks that operate in parallel. Carry-in is estimated in all blocks except the most significant one by using a simplified, low-overhead technique. In the most significant block, carry-in is estimated with higher accuracy using two different paths with different delays. An innovative approach is proposed to select one of these paths for carry generation. Our selection scheme ensures that the shorter path that retains output accuracy is selected based on the values of the input operands. The operation requires two cycles for completion if the longer path is selected for the carry estimation. The selection logic reduces the probability of a two-cycle operation. Therefore, the average delay in SASC is close to the critical path delay of one block. We have evaluated SASC in terms of accuracy, delay, and energy consumption, and compared it with several recently proposed approximate adders. Our contributions are as follows:

- A novel approximate adder with innovative carry prediction and selection logic, offering a high-quality-effort tradeoff, has been proposed.
- The proposed approximate adder features runtime configurability, allowing it to operate either in a high-precision, variable-latency dynamic mode or in a low-precision, fast single-cycle mode.
- The effectiveness of the proposed adder in terms of accuracy and hardware metrics has been thoroughly evaluated and compared with state-of-the-art approximate adders.
- The effectiveness of the proposed adder in real applications, including image dithering algorithms and stock price prediction through regression, has been investigated and confirmed.

The remainder of this paper is organized as follows. Section II presents the related work. Section III introduces the structure of the proposed adder. Section IV investigates the efficiency of the proposed adder compared to several other approximate adders and its applicability in two applications, image dithering and stock price prediction through regression. Finally, Section V concludes the paper.

II. RELATED WORK

Approximate adders can be divided into four categories: 1) adders based on approximate full adders, 2) speculative adders, 3) segmented adders, and 4) approximate carry-select adders [9].

Several studies [10], [11], [12], [13], [14], [15], [16] have altered the structure of conventional full- and half-adders at the transistor or gate level, minimizing their complexity, and consequently, their critical path delay, and power consumption. These approximate half and full adders (AHAs and AFAs) are typically used in the less significant parts of approximate adders, while more significant parts of these adders are still implemented using exact full adders to maintain the adder accuracy within the intended range.

Speculative adders [17], [18], [19], [20], [21] capitalize on the observation that when adding two n -bit integers, the average length of the longest propagate chain is $\log_2(n)$ [17]. Therefore, an n -bit speculative adder uses k bits (where $k < n$) to predict the carry-in at each stage. The advantage of speculative adders is their high speed, achieved at the cost of increased hardware overhead and energy consumption.

Segmented adders [22], [23], [24], [25], [26] consist of multiple sub-adder blocks with independent carry-in that operate in parallel. These adders offer lower hardware complexity and reduced energy consumption while maintaining acceptable levels of accuracy.

Approximate carry-select adders [27], [28], [29], [30], [31], [32] introduce approximations into the carry or sum selection logic of classical carry-select adders. These adders have high hardware overhead but low latency. The following subsections investigate various studies across different categories of approximate adders.

A. AFA-Based Approximate Adders

Various structures have been proposed for AFAs and AHAs including combinations of pass transistor logic and CMOS logic [10], the use of transmission gates [11], modifications to the mirror adder structure at the transistor level [12], the employment of NAND gates [13], and gate-level logic minimization [14]. These approaches result in lower delay and power consumption compared to exact FAs. Some studies have also explored emergent technologies, such as nanoscale spintronic devices, to design AFAs with reduced delay, and power consumption compared to conventional technologies [15], [16].

B. Speculative Adders

A speculative adder called ACA (Almost Correct Adder), proposed in [17], exploits a parallel structure to calculate the

carry-in of each stage using operands from the previous k stages (where $k = \log_2^n$). Another study proposed five speculative adders for high-speed applications by pruning the parallel structures that produce group-generate and group-propagate signals, in common parallel prefix adders, including Brent-Kung, Hybrid Han-Carlson, Sklansky, Lander-Fischer, and Carry increment adders [18]. A method for designing speculative adders by mixing serial and parallel prefix structures was proposed in [19]. This method removes the dependency of error signals and avoids the late completion of operations in the error-correction phase. In other studies [20], [21], an n -bit adder is divided into several 4-bit blocks. Within each block, the carry-in of each stage is computed using the generate and propagate-signals from the previous k -stages ($1 \leq k \leq 4$).

C. Segmented Adders

An approximate segmented adder called Error Tolerant Adder II (ETAIL) was suggested in [22]. ETAIL comprises several blocks, each containing a carry generator and a sum generator. The carry-in to the carry generator of each block is zero. The carry-in of the sum generator in block _{i} is supplied by the carry-out of the carry generator in block _{$i-1$} . A generic accuracy configurable adder (GeAr) was proposed in [23]. GeAr comprises several overlapping adders with a p -bit overlap operating in parallel. It features an error detection and correction mechanism that is activated when the overlapping sections are in propagate mode. Another approximate segmented adder called QuAd, similar in architecture to GeAr, was proposed in [24]. Unlike GeAr, this adder, imposes no restrictions on the size of the sub-adders or the number of overlapping bits, enabling optimization based on the specific needs of each problem. A simplified equal-segment adder (ESA) was presented in [25]. The sub-adder blocks in ESA are non-overlapping, and the carry-in of block _{i} is provided by the MSB of the first or second operand in block _{$i-1$} . ESA offers an excellent quality-effort tradeoff. In [26], a low-power approximate adder was introduced, consisting of two separate RCA segments. The lower segment RCA allows for suppressing input operands and truncating lower bits to all '1', leading to reduced energy consumption at the cost of lower accuracy.

D. Approximate Carry-Select Adders

An approximate adder called the Speculative Carry-Select Adder (SCSA) was proposed in [28]. SCSA comprises several blocks, each consisting of two k -bit sub-adders. One sub-adder assumes that the carry-in is 0 (adder₀), and the other assumes it is "1" (adder₁), computing two values for the output sum, in parallel. In SCSA, the sum output of each block is determined based on the approximate carry-out computed by adder₀ in the preceding block.

Another approximate adder, the Carry Speculative Adder (CSPA), was proposed in [29]. CSPA consists of several blocks, each featuring two carry generators, a multiplexer, a sum generator, and a carry predictor. The carry predictor uses several more significant bits from the block to predict the carry-out. Each block relies on the carry predicted by the preceding block

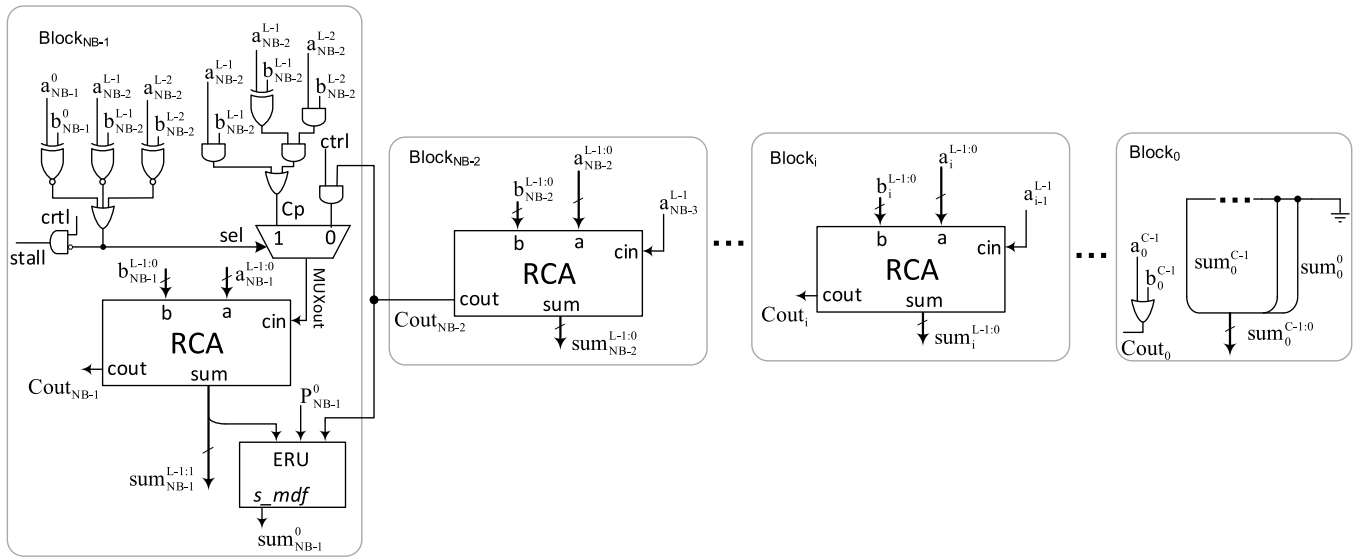


Fig. 1. Internal structure of the proposed adder (SASC).

to select one of its carry generators' outputs, which is then fed into the sum generation logic. In another study [30], a speculative adder was introduced that utilizes generate signals for carry speculation. It features a low-cost error reduction module and a sign correction module to manage relative errors and correct sign errors.

An approximate adder with configurable accuracy, called the simple accuracy-reconfigurable adder (SARA), was presented in [31]. By adjusting several control bits, the carry-in of each block can be connected to either the generate signal corresponding to the MSB or to the carry-out of the preceding block. Therefore, SARA can be configured for high accuracy or low latency. In another study, an approximate adder called the block-based carry speculative approximate adder (BCSA) was proposed [32]. In BCSA, the carry-in of block_i is selected between two approximate candidates: the generate signal corresponding to the MSB in block_{i-1} and the group-generate signal of all bits in block_{i-1}.

III. STRUCTURE OF THE PROPOSED APPROXIMATE ADDER

Fig. 1 shows the structure of the proposed approximate adder, called SASC (Segmented Approximate Adder with Smart Chaining). SASC is denoted by three parameters: SASC (N, L, C), and comprises several blocks. The blocks are named from right (least significant) to left (most significant) as Block₀, Block₁, ..., and Block_{NB-1}, where NB represents the total number of blocks. N represents the bit-width of the adder, C is the bit-width of Block₀, and L is the bit-width of the other blocks. The bit-width of Block₁ may be less than L, depending on the values of N, L, and C. Equations (1) and (2) show how to calculate the number of blocks (NB) and the bit-width of Block₁ (H), respectively.

$$NB = \lceil (N - C)/L \rceil + 1 \quad (1)$$

$$H = N - C - (NB - 2) \times L \quad (2)$$

Block₀ is a constant-truncation block, whose sum output is equal to zero ($sum_0^{C-1:0} = 0$) and its carry-out is produced by an OR operation on the most significant bits of its input operands. The other blocks each contain an L-bit ripple carry adder

(RCA). In other words, each RCA block comprises of L stages. These RCA blocks work in parallel.

In Fact, the N-bit input operands, a and b, are divided into L-bit parts (except for the rightmost part, which includes C bits). Each L-bit part is added by one of the RCA summation blocks. The RCA in Block_i adds the ith group of L bits of the input operands, denoted by $a_i^{L-1:0}$ and $b_i^{L-1:0}$, and computes $sum_i^{L-1:0}$, where i denotes the block number and $L - 1:0$ denotes the bit range of the input operands for that block. Additionally, a_j^i denotes the jth least significant bit of the first operand of block_i.

The carry-in of Block_{*i*}, except for Block_{NB-1} and Block₁, is supplied by the MSB of the first operand in Block_{*i-1*} ($Cin_i = a_{i-1}^{L-1}$). The carry-in of Block₁ is produced by an OR operation on the most significant bits of the input operands to Block₀, denoted by a_0^{C-1} and b_0^{C-1} . The carry-in of the most significant block, Block_{NB-1}, is prepared by a more complex logic depicted in Fig. 1 and discussed later.

As mentioned, the carry-in of Block_i for $1 < i < \text{NB}-1$, is supplied by the most significant bit of the first operand in Block _{$i-1$} . Assuming that operands A and B added together by SASC are uniformly distributed, the probability of correctly estimating the carry-in of Block_i using this scheme is 0.75. For justification, consider adding two uniformly distributed N-bit numbers. If we estimate the carry-out of column_i based on the value of the first operand at this column, Table I illustrates the cases in which this estimation is correct. In this table, a_i and b_i denote the i^{th} bits of the input operands, c_{in} represents the exact value of the carry-in to column_i, $c_{\text{out}_{\text{ex}}}$ is the exact value of the carry-out of column_i, and $c_{\text{out}_{\text{apx}}}$ denotes the approximation of the carry-out of column_i by setting it equal to a_i . The shaded areas represent cases where carry estimation is inaccurate. As shown, carry estimation is accurate in 6 out of 8 cases. Given the uniform distribution of input operands A and B, all rows in Table I have the same probability of occurrence. Therefore, the probability of correctly estimating the carry-out is 0.75.

Each stage of Block_i can be in one of three states, depending on the input values of the stage: generate (G), propagate (P), or

TABLE I
PREDICTED VALUE COMPARED WITH EXACT VALUE OF CARRY OUT

a_i	b_i	c_{in}	$c_{out_{ex}}$	$c_{out_{apx}}$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	0
1	0	0	0	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

kill (K). Equation (3) defines these three states for stage j in Block $_i$, denoted as $Stage^j_i$. Generate and propagate signals for a group of successive stages are defined in (4).

$$G^j_i = a^j_i, b^j_i, P^j_i = a^j_i \oplus b^j_i, K^j_i = (a^j_i)' \cdot (b^j_i)' \quad (3)$$

$$G^{x:x-j}_i = G^x_i + P^x_i \cdot G^{x-1:x-j}_i, P^{x:x-j}_i = \prod_{m=x-j}^x P^m_i \quad (4)$$

Because errors in the more significant bits significantly reduce the accuracy of the adder, the carry-in of the most significant block in SASC is estimated more accurately. The carry-in of the most significant block (Block $_{NB-1}$) is estimated through two different paths: C_p and $C_{out_{NB-2}}$ (see Fig. 1). A MUX selects one of these two values, and the output of the MUX is used as the carry-in for Block $_{NB-1}$ ($C_{in_{NB-1}}$). C_p is the group-generate signal obtained from the two MSBs in Block $_{NB-2}$, generated based on (5). $C_{out_{NB-2}}$ is the carry-out of the adder in Block $_{NB-2}$. The select line of the MUX is asserted according to (6). Equation (6) checks the states of these three stages: $Stage^0_{NB-1}$, $Stage^{L-1}_{NB-2}$, and $Stage^{L-2}_{NB-2}$. If at least one of these stages is not in propagate mode, the MUX places C_p on its output line.

In this situation, the carry-out of the preceding block cannot propagate to the more significant stages in Block $_{NB-1}$, but it can potentially affect the least significant stage of Block $_{NB-1}$. Therefore, a simple estimation of the carry-out of Block $_{NB-1}$ provided by C_p , along with a simple error recovery mechanism, can lead to a desirable accuracy, while keeping the critical path delay close that of one block. If all stages, $Stage^0_{NB-1}$, $Stage^{L-1}_{NB-2}$, and $Stage^{L-2}_{NB-2}$, are in propagate mode, the carry-out of Block $_{NB-2}$ has a chance to propagate to the most significant bits of Block $_{NB-1}$. Therefore, the MUX places $C_{out_{NB-2}}$ on its output line to preserve adder accuracy. However, this operation requires two cycles to complete. During this period, the stall signal is asserted ($stall = sel'$).

$$C_p = G^{L-1:L-2}_{NB-2} = (a^{L-1}_{NB-2} \cdot b^{L-1}_{NB-2}) + (a^{L-1}_{NB-2} \oplus b^{L-1}_{NB-2}) \cdot (a^{L-2}_{NB-2} \cdot b^{L-2}_{NB-2}) \quad (5)$$

$$sel = (p^0_{NB-1})' + (p^{L-1:L-2}_{NB-2})' = (p^0_{NB-1})' + (p^{L-1}_{NB-2})' + (p^{L-2}_{NB-2})' \quad (6)$$

SASC dynamically determines, based on input patterns, whether the two most significant blocks should be chained together or operate in parallel. The stall signal is activated if the last two blocks are chained, extending the addition operation by

one more cycle. According to (6), the addition takes two cycles if $Stage^0_{NB-1}$, $Stage^{L-1}_{NB-2}$ and, $Stage^{L-2}_{NB-2}$ are all in propagate mode. Assuming a uniform distribution of operands, the probability that any stage is in propagate mode is 0.5. Equation (7) indicates the probability that all three mentioned stages are in propagate mode, which corresponds to the probability of a two-cycle operation in SASC.

$$P_{2 \text{ Cycles}} = 0.5^3 = 0.125 \quad (7)$$

According to (7), the average number of execution cycles in SASC is obtained using (8). This confirms that, the carry- selection logic in the most significant block of SASC, keeps the average number of cycles required to complete one operation, close to one cycle.

$$\#Cycles_{avg} = 0.125 \times 2 + (1 - 0.125) \times 1 = 1.125 \text{ Cycles} \quad (8)$$

A lightweight error recovery unit (ERU) is embedded in SASC to reduce the error distance. The ERU modifies sum^0_{NB-1} based on Equation (9).

$$S_mdf = sum^0_{NB-1} + (p^0_{NB-1})' \cdot Cout_{NB-2} \quad (9)$$

To better understand the details of the ERU and carry- selection logic, we discuss the operation of SASC in the following five cases.

Case 1: Suppose that $P^0_{NB-1} = 1$, $P^{L-1:L-2}_{NB-2} = 1$. This means that the least significant bit of Block $_{NB-1}$, along with the two most significant bits of Block $_{NB-2}$, are in propagate mode. As a result, C_p may not accurately represent the value of $C_{out_{NB-1}}$ and this inaccuracy can propagate to the most significant bits of Block $_{NB-1}$. Therefore, the two most significant blocks should be chained to preserve accuracy. Fortunately, as per equation (6), $sel = 0$ and $stall = 1$ which enables the intended chaining. In this case, with the correct chaining, the ERU should not alter the output of Block $_{NB-1}$. With $P^0_{NB-1} = 1$ and according to (9), $S_mdf = sum^0_{NB-1}$, we have achieved our intended goal.

Case 2: Suppose that $P^0_{NB-1} = 1$, $P^{L-1:L-2}_{NB-2} = 0$. According to the condition $P^{L-1:L-2}_{NB-2} = 0$, it implies that at least one of the two stages, either $Stage^{L-1}_{NB-2}$ or $Stage^{L-2}_{NB-2}$, is in generate or kill mode. Consequently, C_p equals $C_{out_{NB-2}}$. Because C_p accurately predicts $C_{out_{NB-2}}$, there is no need to chain the last two blocks, and the ERU should not alter the output of Block $_{NB-1}$. Equation (6) dictates that $sel = 1$, and C_p is placed on the MUX output (see Fig. 2). Moreover, $stall = sel' = 0$, ensuring that the operation completes in one cycle, which is the correct decision. Additionally, with $P^0_{NB-1} = 1$ and according to (9), $S_mdf = sum^0_{NB-1}$. Consequently, the ERU does not change the output, aligning with our intention.

Case 3: Suppose that $P^0_{NB-1} = 0$, $P^{L-1:L-2}_{NB-2} = 0$. As with Case 2, the condition $P^{L-1:L-2}_{NB-2} = 0$ indicates that either $Stage^{L-1}_{NB-2}$ or $Stage^{L-2}_{NB-2}$, is in generate or kill mode and C_p correctly predicts $C_{out_{NB-2}}$, meaning $C_p = C_{out_{NB-2}}$. Therefore, there is no need to chain the two most significant blocks, and the ERU should not alter the output of Block $_{NB-1}$. According to (6), $sel = 1$, and $stall = 0$. Thus, the two most significant blocks are not chained together, and the operation completes in one cycle, which aligns

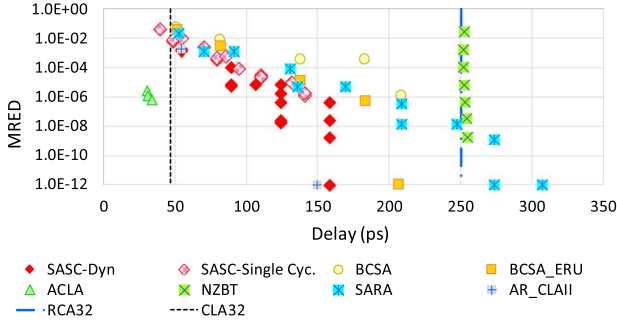


Fig. 2. MRED-delay of the 32-bit configurations of the investigated approximate adders.

with our intention. Additionally, with $P_{NB-1}^0 = 0$, it implies that $Stage_{NB-1}^0$ is in generate or kill mode. Therefore, the sum bit of this stage equals the value of its carry-in, which can be expressed as $sum_{NB-1}^0 = C_p = Cout_{NB-2}$. Considering (9), we can formulate equation (10) as follows, confirming that the ERU does not interfere with the output of the most significant block, thus aligning with our intention.

$$\begin{aligned}
 S_mdf &= sum_{NB-1}^0 + (P_{NB-1}^0)' \cdot Cout_{NB-2} \\
 &= sum_{NB-1}^0 + Cout_{NB-2} \\
 &= sum_{NB-1}^0 + sum_{NB-1}^0 \\
 &= sum_{NB-1}^0
 \end{aligned} \quad (10)$$

Case 4: Suppose that $P_{NB-1}^0 = 0$, $P_{NB-2}^{L-1:L-2} = 1$, $Cout_{NB-2} = 0$. In this scenario, since $P_{NB-1}^0 = 0$, the carry-in of the most significant block does not propagate through this block. Therefore, we can utilize C_p as the carry-in, allowing the operation to complete in one cycle. Referring to (6), we conclude that $sel = 1$, and $stall = 0$, which aligns perfectly with the intended operation. Additionally, given $P_{NB-2}^{L-1:L-2} = 1$, and referring to equation (5), we can conclude that $C_p = 0$, which also implies $C_p = Cout_{NB-2} = 0$. Consequently, the ERU should not alter the output of Block_{NB-1}. Equation (9) leads us to derive (11), confirming that the output of the most significant block remains unaffected by the ERU in this case.

$$\begin{aligned}
 S_mdf &= sum_{NB-1}^0 + (P_{NB-1}^0)' \cdot Cout_{NB-2} \\
 &= sum_{NB-1}^0 + Cout_{NB-2} \\
 &= sum_{NB-1}^0 + 0 = sum_{NB-1}^0
 \end{aligned} \quad (11)$$

Case 5: Suppose that $P_{NB-1}^0 = 0$, $P_{NB-2}^{L-1:L-2} = 1$, $Cout_{NB-2} = 1$. Similar to the previous case, because $P_{NB-1}^0 = 0$, the carry-in of the most significant block does not propagate through this block. Therefore, C_p can be used as the carry-in for Block_{NB-1}, allowing the operation to complete in one cycle. According to equation (6), $sel = 1$, and $stall = 0$, which is in accordance with our intended operation. Given the condition, $P_{NB-2}^{L-1:L-2} = 1$ and equation (5), we deduce that $C_p = 0$ implying $C_p \neq Cout_{NB-2}$. This misprediction of the carry-in for the most significant block, can affect the least significant output bit of this block but cannot propagate to the most significant bits, thanks to $P_{NB-1}^0 = 0$. As a result, the ERU should correct sum_{NB-1}^0

which incorrectly equals 0. This correction is achieved as per equation (12).

$$\begin{aligned}
 S_mdf &= sum_{NB-1}^0 + (P_{NB-1}^0)' \cdot Cout_{NB-2} \\
 &= sum_{NB-1}^0 + Cout_{NB-2} \\
 &= sum_{NB-1}^0 + 1 = 1
 \end{aligned} \quad (12)$$

As shown in Fig. 1, SASC has an input signal named ctrl, which allows it to operate in either dynamic or single-cycle mode. When ctrl is not asserted, SASC operates in single cycle mode, where the most significant block is not chained to its previous block, the stall signal is not asserted and the operation completes in one cycle. Otherwise, when ctrl is asserted, SASC operates in dynamic mode, where the operation can extend to two cycles based on the values of the input operands. The single-cycle mode is faster but sacrifices some accuracy. This mode of operation is suitable when high accuracy is not required or when variable latency cannot be tolerated.

IV. RESULTS AND DISCUSSION

We described SASC, and six other recently proposed approximate adders, SARA [31], BCSA [32], BCSA_ERU [32], ACLA [20], NZBT [26], and AR_CLAII [21], in synthesizable Verilog. The normalized mean error distance (NMED) and mean relative error distance (MRED) for all adders were obtained using Monte Carlo simulations on Verilog models for 10 million input patterns. For SASC, BCSA, and BCSA_ERU, which have variable delays depending on the input patterns, the average number of cycles required to perform one operation was obtained from the RTL simulation.

The Verilog models of the approximate adders were synthesized on *NanGate 15 nm Cell Library*. The Verilog models were described structurally, and we instructed the synthesis tool to maintain the structure and avoid combining sub-modules. We did not impose specific constraints on timing or power optimization.

All post-synthesis simulations were performed at a supply voltage of 0.8 V and a temperature of 25°C. For power analysis, post-synthesis simulations were conducted with 1 million input patterns, and the signal activity of all internal connections of each model was saved in VCD files. The signal activities were back-annotated in the synthesized models, and the average power consumed by each adder was obtained. Other hardware metrics, including delay, and energy per operation, were extracted for all the adders. In the following subsections, we assess SASC and compare it with SARA, BCSA, BCSA_ERU, ACLA, AR_CLAII, and NZBT in terms of accuracy, delay, and energy consumption. In addition, the efficiency of SASC in performing image dithering algorithm and in stock price prediction by regression was evaluated and compared with other approximate adders.

A. Error Metrics Evaluations

We simulated the synthesizable Verilog model of SASC and several other approximate adders using 10 million pairs of random inputs and computed the error rate (ER), normalized mean error distance (NMED), and mean relative error distance (MRED). NMED and MRED are obtained for an N-bit approximate adder

TABLE II
ERROR METRICS AND AVERAGE CYCLES FOR 16-BIT SASC

SASC Config. (N-L-C)	ER	NMED	MRED	C _{avg}
16-2-0 (Dyn)	82.172	1.9E-02	2.5E-02	1.1244
16-2-0 (Single Cyc)	83.244	3.2E-02	4.0E-02	1.0000
16-4-0 (Dyn)	43.566	1.0E-03	1.4E-03	1.1250
16-4-0 (Single Cyc)	47.120	4.8E-03	6.5E-03	1.0000
16-6-0 (Dyn)	24.966	6.1E-05	8.3E-05	1.1248
16-6-0 (Single Cyc)	29.582	1.0E-03	1.4E-03	1.0000
16-8-0 (Dyn)	0.000	0	0	1.1248
16-8-0 (Single Cyc)	6.206	2.4E-04	3.3E-04	1.0000
16-2-2 (Dyn)	94.125	1.9E-02	2.5E-02	1.1244
16-2-2 (Single Cyc)	94.475	3.2E-02	4.0E-02	1.0000
16-4-2 (Dyn)	85.940	1.0E-03	1.4E-03	1.1250
16-4-2 (Single Cyc)	86.826	4.9E-03	6.5E-03	1.0000
16-6-2 (Dyn)	81.162	7.3E-05	9.9E-05	1.1248
16-6-2 (Single Cyc)	82.344	1.0E-03	1.4E-03	1.0000
16-8-2 (Single Cyc)	74.910	1.7E-05	2.3E-05	1.1248
16-8-2 (Single Cyc)	76.473	2.7E-04	3.6E-04	1.0000
16-2-4 (Dyn)	98.052	1.9E-02	2.5E-02	1.1244
16-2-4 (Single Cyc)	98.167	3.2E-02	4.0E-02	1.0000
16-4-4 (Dyn)	95.353	1.0E-03	1.4E-03	1.1250
16-4-4 (Single Cyc)	95.660	4.9E-03	6.5E-03	1.0000
16-6-4 (Dyn)	93.771	7.8E-05	1.1E-04	1.1248
16-6-4 (Single Cyc)	94.183	1.1E-03	1.4E-03	1.0000
16-8-4 (Dyn)	93.771	7.8E-05	1.1E-04	1.1248
16-8-4 (Single Cyc)	94.242	3.5E-04	4.7E-04	1.0000
16-2-6 (Dyn)	99.347	1.9E-02	2.5E-02	1.1244
16-2-6 (Single Cyc)	99.390	3.2E-02	4.0E-02	1.0000
16-4-6 (Dyn)	98.855	1.2E-03	1.7E-03	1.1250
16-4-6 (Single Cyc)	98.932	5.1E-03	6.8E-03	1.0000
16-6-6 (Dyn)	98.471	3.2E-04	4.4E-04	1.1248
16-6-6 (Single Cyc)	98.585	1.4E-03	1.9E-03	1.0000
16-8-6 (Dyn)	98.471	3.2E-04	4.4E-04	1.1248
16-8-6 (Single Cyc)	98.653	6.8E-04	9.3E-04	1.0000
16-2-8 (Dyn)	99.789	1.9E-02	2.5E-02	1.1244
16-2-8 (Single Cyc)	99.804	3.2E-02	4.0E-02	1.0000
16-4-8 (Dyn)	99.623	1.3E-03	1.8E-03	1.1250
16-4-8 (Single Cyc)	99.650	5.7E-03	7.6E-03	1.0000
16-6-8 (Dyn)	99.623	1.3E-03	1.8E-03	1.1248
16-6-8 (Single Cyc)	99.666	2.7E-03	3.7E-03	1.0000
16-8-8 (Dyn)	99.805	1.9E-03	2.5E-03	1.0000
16-8-8 (Single Cyc)	99.805	1.9E-03	2.5E-03	1.0000

TABLE III
ERROR METRICS AND AVERAGE CYCLES FOR 32-BIT SASC

SASC Config. (N-L-C)	ER	NMED	MRED	C _{avg}
32-4-0 (Dyn)	82.050	1.0E-03	1.4E-03	1.1247
32-4-0 (Single Cyc)	82.774	4.8E-03	6.4E-03	1.0000
32-8-0 (Dyn)	43.711	3.8E-06	5.2E-06	1.1260
32-8-0 (Single Cyc)	47.216	2.5E-04	3.4E-04	1.0000
32-12-0 (Dyn)	24.853	1.5E-08	1.8E-08	1.1253
32-12-0 (Single Cyc)	29.606	1.5E-05	2.2E-05	1.0000
32-16-0 (Dyn)	0.000	0	0	1.1244
32-16-0 (Single Cyc)	6.175	9.4E-07	1.3E-06	1.0000
32-4-4 (Dyn)	98.543	1.0E-03	1.4E-03	1.1247
32-4-4 (Single Cyc)	98.824	4.8E-03	6.4E-03	1.0000
32-8-4 (Dyn)	96.519	3.8E-06	5.3E-06	1.1260
32-8-4 (Single Cyc)	96.975	2.5E-04	3.4E-04	1.0000
32-12-4 (Single Cyc)	95.353	1.6E-08	1.9E-08	1.1253
32-12-4 (Single Cyc)	95.629	1.5E-05	2.2E-05	1.0000
32-16-4 (Dyn)	93.771	1.2E-09	1.6E-09	1.1244
32-16-4 (Single Cyc)	94.122	9.4E-07	1.3E-06	1.0000
32-4-8 (Dyn)	99.885	1.0E-03	1.4E-03	1.1247
32-4-8 (Single Cyc)	99.966	4.8E-03	6.4E-03	1.0000
32-8-8 (Dyn)	99.866	3.8E-06	5.3E-06	1.1260
32-8-8 (Single Cyc)	99.756	2.5E-04	3.4E-04	1.0000
32-12-8 (Dyn)	99.623	2.0E-08	2.6E-08	1.1253
32-12-8 (Single Cyc)	99.653	1.5E-05	2.2E-05	1.0000
32-16-8 (Dyn)	99.623	2.0E-08	2.6E-08	1.1244
32-16-8 (Single Cyc)	99.646	9.7E-07	1.3E-06	1.0000
32-4-12 (Dyn)	99.991	1.0E-03	1.4E-03	1.1247
32-4-12 (Single Cyc)	99.996	4.8E-03	6.4E-03	1.0000
32-8-12 (Dyn)	99.985	4.0E-06	5.6E-06	1.1260
32-8-12 (Single Cyc)	99.987	2.5E-04	3.4E-04	1.0000
32-12-12 (Dyn)	99.977	3.2E-07	4.5E-07	1.1253
32-12-12 (Single Cyc)	99.978	1.6E-05	2.3E-05	1.0000
32-16-12 (Dyn)	99.977	3.2E-07	4.5E-07	1.1244
32-16-12 (Single Cyc)	99.980	1.4E-06	1.9E-06	1.0000
32-4-16 (Dyn)	99.998	1.0E-03	1.4E-03	1.1247
32-4-16 (Single Cyc)	99.999	4.8E-03	6.4E-03	1.0000
32-8-16 (Dyn)	99.998	5.1E-06	7.1E-06	1.1260
32-8-16 (Single Cyc)	99.998	2.5E-04	3.5E-04	1.0000
32-12-16 (Dyn)	99.998	5.1E-06	7.1E-06	1.1253
32-12-16 (Single Cyc)	99.998	2.2E-05	3.2E-05	1.0000
32-16-16 (Dyn)	99.999	7.3E-06	1.0E-05	1.0000
32-16-16 (Single Cyc)	99.999	7.3E-06	1.0E-05	1.0000

using (13) and (14), where R_i and R'_i denote the exact sum, and the approximate sum for sample i , respectively, n represents the number of random samples, and N is the bit-width of the adder.

$$NMED = \frac{1}{2^N - 1} \sum_{i=1}^n \frac{|R_i - R'_i|}{n} \quad (13)$$

$$MRED = \frac{1}{n} \sum_{i=1}^n \frac{|R_i - R'_i|}{R_i} \quad (14)$$

For SASC, BCSA, and BCSA_ERU, whose delays vary with input patterns, we simulated their Verilog models with 10 million pairs of random inputs, and obtained the average number of cycles required for each operation, denoted as C_{avg} . The average delay of these adders, reported in the following subsections, is given by (15), where t_c represents the cycle duration of the adder, which is extracted from post-synthesis reports.

$$Delay_{avg} = t_c \times C_{avg} \quad (15)$$

Tables II and III present the error metrics and the average number of cycles for different configurations of the 16-bit and 32-bit

SASC respectively. The different configurations are denoted by the parameters N , L and C . As discussed in Section II, N is the bit-width of the adder, L is the bit-width of the SASC blocks (except for the least significant one), and C indicates the bit-width of the least significant block. Due to the variety of configurations for the approximate adders, Tables II and III report only the results for SASC. However, the scatter plots in the following subsection facilitate the comparison of SASC with other approximate adders in terms of accuracy and hardware metrics. The error metrics for SASC are reported for both dynamic and single-cycle modes of operation. The results presented in Tables II and III show that as L increases, the error metrics improve due to the increase in the number of bits contributing to carry generation. Increasing C has a negligible effect on the accuracy drop, particularly for smaller values of L , indicating that the least significant bits of the result have little impact on the error metrics.

The average number of cycles (C_{avg}) required to complete a single operation is reported in Tables II and III. The simulation results indicate that, for all configurations of SASC except one, C_{avg} is very close to 1.125 cycles in dynamic mode, confirming (8). For one configuration of each bit-length, the average

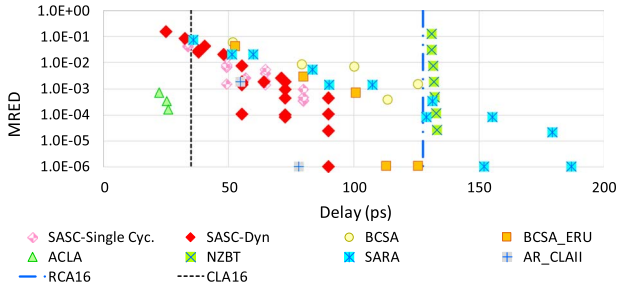


Fig. 3. MRED-delay of the 16-bit configurations of the investigated approximate adders.

number of cycles equals one. This is because these configurations consist of only one block, and block chaining is therefore not possible. In these configurations, the carry-in of the single block is provided through the group generate signal of two preceding bit positions.

Our simulations also indicate that C_{avg} is 1.99, 1.91, 1.81, and 1.56 cycles for BCSA and BCSA_ERU, corresponding to configurations with $L = 4, 8, 12$, and 16 , respectively. As a result, the average delay for BCSA and BCSA_ERU exceeds that of SASC, a topic discussed in more detail in Section V.B. In BCSA and BCSA_ERU, an operation takes two cycles if the carry is incorrectly estimated in any block, while in SASC, it requires two cycles only if the carry estimate for the most significant block is incorrect. Consequently, the likelihood of an operation taking two cycles is higher in BCSA and BCSA_ERU than in SASC.

B. Evaluation of Hardware Design Metrics

In this section, the proposed approximate adder, SASC, is compared with six other approximate adders: SARA [31], NZBT [26], ACLA [20], BCSA [32], BCSA_ERU [32], and AR_CLAII [21], in terms of MRED/delay, MRED/energy, and MRED/EDP (Energy-Delay-Product). The MRED values are represented on a logarithmic scale, and configurations with MRED values of zero placed at the bottom of the graphs along the x-axis. The characteristics of the RCA and CLA exact adders are represented by vertical lines in these graphs.

Figs. 2 and 3 depict the MRED values for 32-bit and 16-bit configurations of the approximate adders in terms of delay. As shown, ACLA configurations exhibit the lowest delay among all approximate adders. However, ACLA lacks high precision (low MRED) configurations. Figs. 4 and 5 illustrate MRED versus energy consumption for 32-bit and 16-bit configurations, respectively. Our results confirm that while ACLA configurations have the lowest delay, they demonstrate high energy consumption due to the highly parallel structure of the carry generation logic.

After ACLA and AR_CLAII, SASC demonstrates the best performance in terms of MRED-Delay. However, SASC performs better than ACLA and AR_CLAII in terms of MRED/Energy and MRED/Delay. In 32-bit configurations, when MRED values are less than $1.0E-2$, SASC outperforms the fastest configurations of BCSA, BCSA_ERU, SARA, and NZBT by 33.9%, 34.3%, 23.0%, and 78.2%, respectively, all while maintaining a lower MRED. For 16-bit configurations, SASC

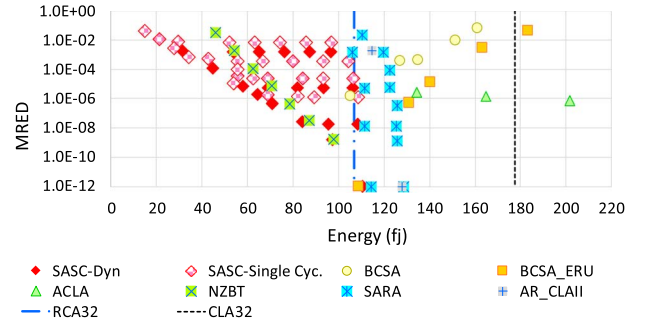


Fig. 4. MRED-energy of the 32-bit configurations of the investigated approximate adders.

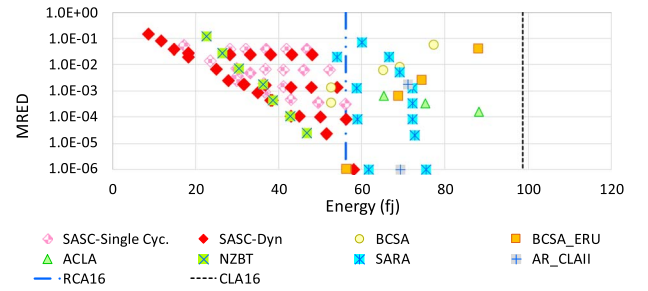


Fig. 5. MRED-energy of the 16-bit configurations of the investigated approximate adders.

achieves a speed advantage of 30.5%, 30.9%, 33.9%, and 57.9% over the fastest setups of BCSA, BCSA_ERU, SARA, and NZBT, respectively, without compromising accuracy. The exact configuration of SASC (MRED = 0), is on average, 36.6% and 29.7% faster than RCA, for 32-bit and 16-bit configurations, respectively. This speed advantage is attributed to SASC's segmented architecture, which reduces the length of the carry chain, and its efficient carry prediction and error recovery logic to preserve accuracy.

BCSA and BCSA_ERU exhibit lower speeds compared to SASC due to a higher probability of two-cycle operations, leading to increased average delays. Our simulation results illustrate that in SASC, the probability of a two-cycle operation is 0.125, whereas in BCSA and BCSA_ERU, this probability ranges from 0.56 to 0.99 depending on the configuration. SASC triggers a two-cycle operation only when inaccurate carry prediction occurs in the most significant block, while any inaccuracy in predicting carry-in values for any block in BCSA and BCSA_ERU results in a two-cycle operation.

For higher precisions, SARA has larger delay values than SASC because it always chains its blocks to increase accuracy, regardless of the value of the input operands. In contrast, SASC exhibits a dynamic behavior for chaining the last two blocks based on the input operands. The high delay values of NZBT at high accuracies are because NZBT requires a long RCA at the more significant bits to enhance accuracy.

Figs. 4 and 5 illustrate MRED and the energy consumed per operation for the studied adders, for 32-bit and 16-bit configurations, respectively. SASC exhibits the best MRED-Energy performance for 32-bit configurations in both dynamic and single-cycle modes. In 16-bit configurations, NZBT outperforms SASC for

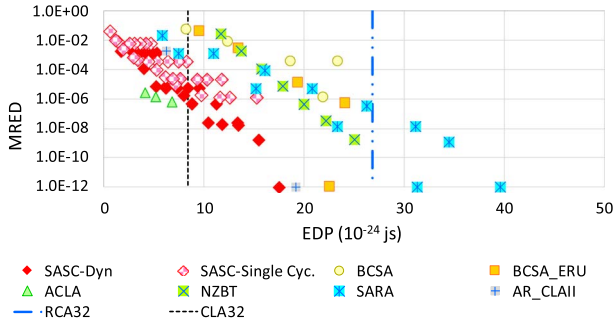


Fig. 6. MRED-EDP of the 32-bit configurations of the investigated approximate adders.

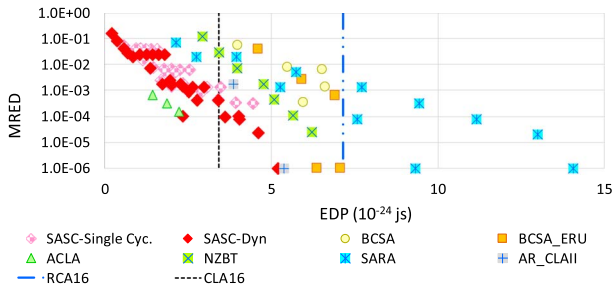


Fig. 7. MRED-EDP of the 16-bit configurations of the investigated approximate adders.

low MRED values. However, SASC surpasses NZBT in terms of delay and also energy-delay product (EDP). Most SASC configurations consume less energy than RCA. In exact mode, SASC consumes only 3.4% more energy than RCA in both 32-bit and 16-bit configurations.

Compared with the least energy-consuming 32-bit configurations of BCSA, ACLA, SARA, NZBT, and AR_CLAII, SASC consumes 32.5%, 52.0%, 57.7%, 31.7%, and 72.5% less energy, respectively, without compromising accuracy. SASC consumes 18.7%, 41.7%, 53.8%, 34.1%, and 16.1% less energy, with an equal or less MRED, compared with the least energy-consuming 16-bit configurations of BCSA, ACLA, SARA, NZBT, and AR_CLAII. The least energy-consuming 32-bit and 16-bit BCSA_ERU configurations consume 1.4% and 2.8% less energy compared to SASC with a comparable MRED, respectively.

The poor performance of ACLA in terms of MRED-Energy is due to its large parallel structure of carry generation logic, even though this logic minimizes its delay. The high energy consumption of BCSA and BCSA_ERU is due to the large carry generators incorporated in each block to provide a more accurate estimate of carry-in values when the operation is extended to two cycles.

Figs. 6 and 7 display MRED in terms of the energy-delay product (EDP) for 32-bit and 16-bit configurations of approximate adders, respectively. ACLA and SASC outperform the other approximate adders in terms of MRED-EDP. Although ACLA surpasses SASC in terms of MRED-EDP, it comes at the cost of significantly higher energy consumption. When compared to the 32-bit configurations of BCSA, BCSA_ERU, SARA, NZBT, and AR_CLAII, which have the lowest EDP

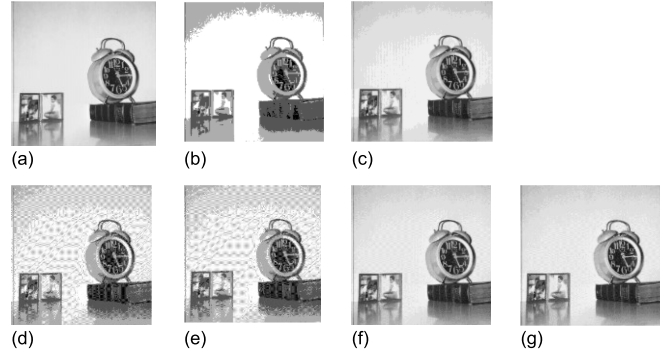


Fig. 8. Transforming an image from a palette with 65536 grayscale shades to another palette with 4 or 16 grayscale shades. (a) Original image (16-bit color depth). (b) Quantized image without dithering (2-bit color depth). (c) Quantized image without dithering (4-bit color depth). (d) Dithering with an exact adder (2-bit color depth). (e) Dithering with SASC-16-6-4 (2-bit color depth). (f) Dithering with an exact adder (4-bit color depth). (g) Dithering with SASC-16-6-4 (4-bit color depth).

values, SASC achieves improvements of 79.0%, 81.9%, 70.1%, 85.1%, and 72.1% in EDP, respectively, without sacrificing MRED. For the 16-bit configurations, the respective improvements are 83.0%, 85.3%, 68.2%, 87.4%, and 54.9%.

Compared to the most accurate (lowest MRED) 32-bit configurations of BCSA, BCSA_ERU, SARA, NZBT, and AR_CLAII, SASC improves EDP by 59.8%, 22.3%, 44.0%, 38.2%, and 8.4%, respectively, while maintaining an equal or lower MRED. The corresponding improvements for the 16-bit configurations are also 60.5%, 18.4%, 44.3%, 25.8%, and 3.3%.

It is worth noting that in single-cycle mode and at higher MRED values, SASC shows the highest performance in terms of MRED/Delay, MRED/Energy, and MRED/EDP in both 16-bit and 32-bit configurations.

C. Runtime Configurability

Runtime configurability can be beneficial in systems with dynamic accuracy requirements, allowing them to adjust accuracy levels in exchange for reduced energy consumption. Among the approximate adders experimented with, SARA [31], ACLA [20], AR_CLAII [21], NZBT [26], and the proposed adder SASC (via the ctrl signal to operate in either one or two cycles) feature some level of runtime configurability. According to the results presented in Figs. 4 and 5, the most accurate configuration of SASC consumes less energy compared to all configurations of ACLA, AR_CLAII, and SARA, except for one configuration of SARA, which consumes 2.3% and 3.5% less energy than SASC in the 32-bit and 16-bit configurations, respectively. This makes SASC an excellent choice for a wide range of applications with different accuracy requirements. Compared with SASC, NZBT offers a wider range of energy configurability but at the cost of significantly higher energy consumption at lower accuracy levels, with up to 3.1x and 2.3x higher energy consumption. In applications with widely varying accuracy requirements, NZBT performs better than SASC. However, for applications with a predetermined accuracy requirement, SASC demonstrates better energy-accuracy efficiency.

TABLE IV
PSNR OF IMAGES AFTER IMAGE DITHERING

Adder	Eng. (fj)	EDP (10^{-24} JS)	Elaine	Boat	Tulips	PSNR Mandrill	Peppers	Clock	Couple	Min
SASC ¹	35.81	1.88	32.30	32.17	33.82	32.61	32.02	30.11	32.24	30.11
SASC ²	38.06	3.41	38.21	38.65	40.97	38.20	37.53	33.27	38.55	33.27
BCSA	65.25	6.54	30.25	30.09	30.15	30.31	30.45	29.54	30.35	29.54
BCSA_ERU	68.86	6.93	30.86	30.79	30.48	30.69	30.78	29.68	30.86	29.68
ACLA	65.26	1.43	32.67	32.49	32.72	32.27	33.81	30.43	32.66	30.43
SARA	58.64	5.26	31.94	31.70	33.66	32.12	31.65	29.72	31.85	29.72
NZBT	36.21	4.76	31.91	31.78	33.29	32.37	31.99	30.16	31.71	30.16
AR_CLAII	71.07	3.86	35.34	35.58	32.90	32.86	33.49	35.09	35.57	32.86
RCA	56.10	7.16					-			
CLA	98.71	3.44					-			

SASC¹: N = 16, L = 4, C = 6
 SASC²: N = 16, L = 8, C = 6
 BCSA & BCSA_ERU: N = 16, L = 6

ACLA: ACLAI configuration
 SARA: N = 16, L = 4, Mode = 2 (at most two consecutive blocks are chained)

NZBT: N = 16, L = 8
 AR_CLA: AR_CLAII configuration

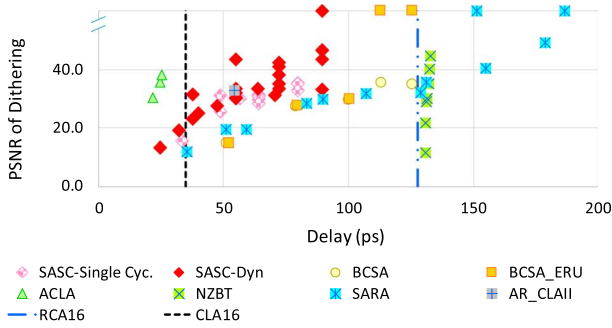


Fig. 9. PSNR-delay of the 16-bit configurations of the investigated approximate adders in image dithering.

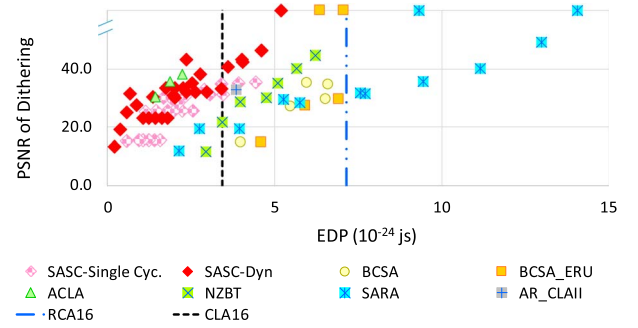


Fig. 11. PSNR-EDP of the 16-bit configurations of the investigated approximate adders in image dithering.

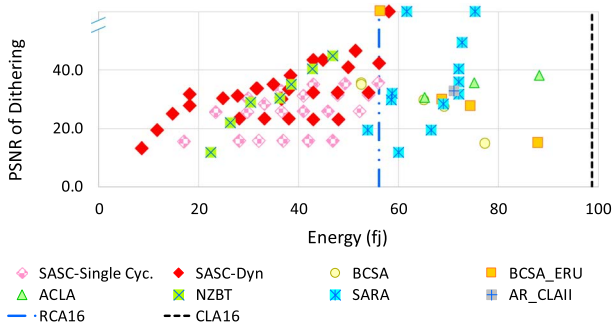


Fig. 10. PSNR-energy of the 16-bit configurations of the investigated approximate adders in image dithering.

D. Quality Analysis in Image Processing

To evaluate the effectiveness of the proposed adder in image processing applications, we utilized SASC and other approximate adders for image dithering using the Floyd–Steinberg dithering algorithm. Image dithering is employed to transform an image from one color space to another, reducing the number of color shades while also preventing color banding. This is particularly useful for accurately displaying images on hardware capable of rendering only a limited range of colors. Dithering is also useful in image size reduction.

The Floyd–Steinberg dithering algorithm operates on the principle of error diffusion, where the quantization error of each pixel is distributed to several neighboring pixels, influencing

their quantization. Fig. 8 depicts a sample image on which we have applied the dithering algorithm. The original image is a 16-bit grayscale image. We quantized the image to both 2-bit and 4-bit grayscale versions, once without dithering and then by applying the Floyd–Steinberg dithering algorithm. The dithering was performed once using an exact adder and then with a sample configuration of SASC (N = 16, L = 6, and C = 4). As shown in the figure, the outputs of SASC and the exact adder are very similar and are not distinguishable.

We applied the dithering algorithm to seven 16-bit grayscale images, each sized 256×256 pixels. These images include Boat, Elaine, Tulips, Mandrill, Peppers, Clock, and Couple. The transformation involved converting these images from a 16-bit grayscale palette to a 4-bit palette. The dithering algorithm was applied to these images in various experiments using both an exact adder and the 16-bit approximate adders. We measured the Peak Signal-to-Noise Ratio (PSNR) of the output images generated by the approximate adders in comparison with the output image of the exact adder. Table IV presents the PSNR values of the dithered images for a sample configuration of each approximate adder. For each approximate adder, among the different configurations, we selected the configuration with the minimum EDP whose output image closely matched the output image of the exact adder in terms of human perception. As shown in Table IV, SASC demonstrates the best performance in terms of energy consumption for image dithering. Table IV indicates that SASC, with the lowest EDP and energy consumption,

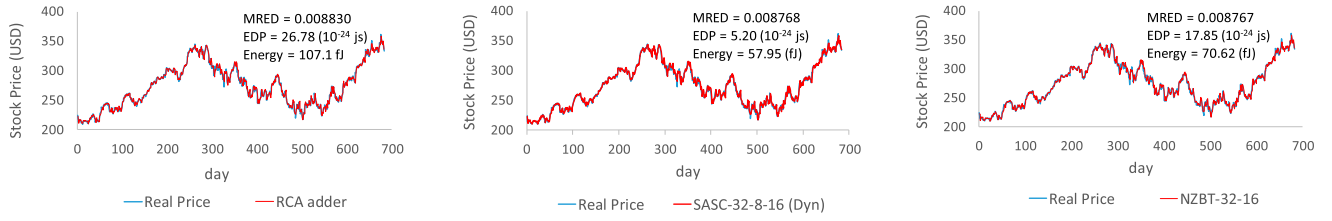


Fig. 12. Stock price prediction by sample 32-bit configurations of the studied approximate adders.

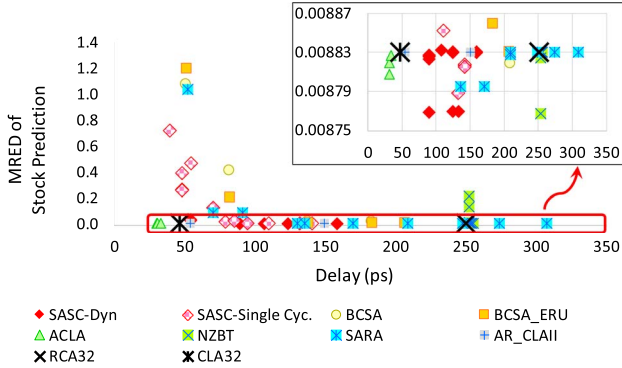


Fig. 13. MRED-delay of stock price prediction by 32-bit configurations of the investigated approximate adders.

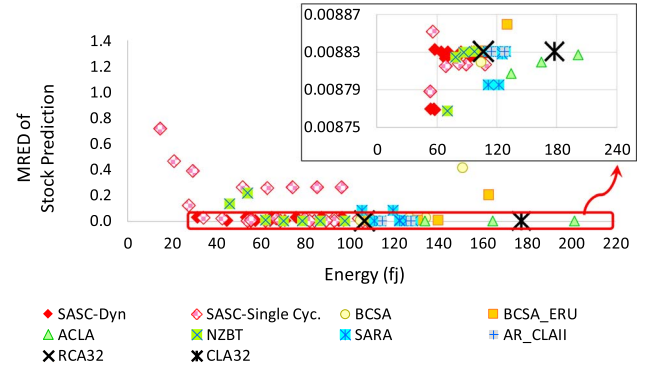


Fig. 14. MRED-energy of stock price prediction by 32-bit configurations of the investigated approximate adders.

generates images with higher PSNR than the other approximate adders, except for ACLA. However, the low EDP for ACLA comes at the cost of significantly higher energy consumption. For a more comprehensive comparison, Figs. 9, 10, and 11 depict PSNR values of the images processed by all configurations of the approximate adders, in terms of delay, energy, and EDP, respectively. Each reported PSNR in these figures is the minimum value of PSNR for all images processed by a configuration of an approximate adder.

Figs. 9, 10, and 11 show that, excluding ACLA, the proposed adder performs the best in executing the dithering algorithm in terms of delay, energy, and EDP. Although ACLA performs better than SASC in terms of delay, it significantly consumes more energy than SASC.

E. Quality Analysis in Stock Price Prediction

To further evaluate the effectiveness of the proposed approximate adder, we employed 32-bit configurations of both the proposed adder and several other studied approximate adders to predict the stock price of Microsoft Corporation using regression. We obtained Microsoft Corporation's stock price history for 3,416 days, from 2010 to 2023 from Yahoo Finance. We used 80% of this data as training data to develop a regression model for predicting stock prices. To make predictions, we utilized the open, high, low, and close (OHLC) signal values from the previous 30 days to forecast the opening stock price for the next day. Additionally, we reserved data from 683 days (20%) for assessing the accuracy of our predictive model. In these experiments, we conducted tests using both exact and approximate adders to perform the addition operations within the regression model.

Fig. 12 illustrates the predictions obtained using the RCA exact adder, as well as those from the most accurate configurations of

SASC and NZBT as samples. As shown in the figure, the sample configurations of SASC and NZBT provide slightly more accurate predictions (lower MRED) compared to the exact adder, with a significant reduction in energy and EDP. SASC outperforms the exact adder by 45.8% and 80.5% in terms of energy consumption and EDP, respectively. Compared to NZBT, SASC demonstrates a 17.9% and 70.8% reduction in energy consumption and EDP, respectively.

Figs. 13, 14, and 15 respectively show MRED/delay, MRED/energy, and MRED/EDP for various configurations of the approximate adders in the context of stock price prediction using regression. Each figure includes a zoomed area for greater clarity. The figures demonstrate that SASC outperforms the other studied approximate adders in terms of delay, except for ACLA and AR_CLAII, and outperforms all the approximate adders including ACLA and AR_CLAII in terms of energy, and EDP. Among the fastest configurations of the approximate adders, SASC provides improvements of 21.7%, 23.0%, 23.5%, and 64.5% in delay over BCSA, BCSA_ERU, SARA, and NZBT respectively, while preserving prediction accuracy.

Fig. 14 demonstrates that SASC outperforms BCSA, BCSA_ERU, ACLA, SARA, NZBT, and AR_CLAII by 45.0%, 46.9%, 56.9%, 70.2%, 31.7%, and 49.6%, respectively, in terms of energy consumption, compared to their configurations with the least energy consumption. Importantly, this improvement in energy efficiency is achieved without any loss in accuracy.

In terms of EDP, when comparing SASC with the configurations of BCSA, BCSA_ERU, SARA, NZBT, and AR_CLAII that have the lowest EDP values, SASC exhibits improvements of 79.0%, 74.8%, 70.1%, 85.1%, and 16.9% in EDP, respectively, all while maintaining accuracy. The ACLA configuration with the lowest EDP has a slight advantage over SASC in terms

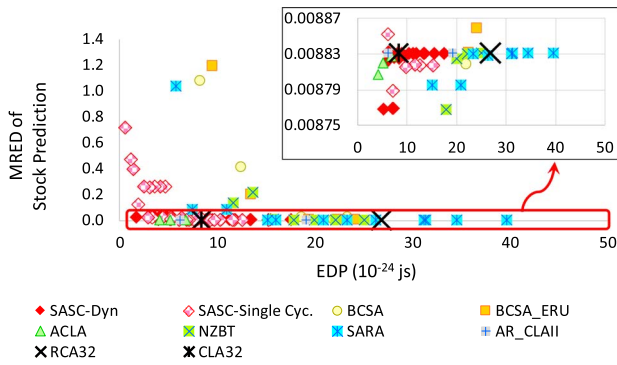


Fig. 15. MRED-EDP of stock price prediction by 32-bit configurations of the investigated approximate adders.

of EDP, but this comes at the cost of significantly higher energy consumption.

V. CONCLUSION

In this study, we proposed an approximate adder called the segmented approximate adder with smart chaining (SASC). The proposed adder comprises several non-overlapping blocks operating in parallel. The rightmost (least significant) block performs constant truncation. The other blocks each contain an RCA, whose carry-in is supplied by the MSB of one of the operands from the earlier block. In the most significant block, carry prediction is performed using two approaches, one of which is more accurate and extends the operation to two cycles. Carry-selection for the most significant block is performed dynamically according to the value of the input patterns. This carry-selection mechanism minimizes the possibility of a two-cycle operation, while maintaining the accuracy of the adder. An error recovery mechanism is embedded in the most significant block to further enhance accuracy. Our investigations into the accuracy of SASC, considering design metrics such as delay, energy, and energy-delay product (EDP), show that the proposed adder offers a better quality-effort tradeoff than the other approximate adders. Compared with several recently proposed approximate adders, different configurations of SASC can improve delay, energy consumption, and EDP by up to 78%, 72%, and 87%, respectively. We also demonstrated the effectiveness of the proposed adder in implementing the image-dithering algorithm, and in stock price prediction using regression.

REFERENCES

- [1] M. Shafique, R. Hafiz, S. Rehman, W. El-Harouni, and J. Henkel, "Invited: Cross-layer approximate computing: From logic to architectures," in *Proc. 53rd ACM/EDAC/IEEE Des. Automat. Conf. (DAC)*, 2016, pp. 1–6.
- [2] Q. Xu, T. Mytkowicz, and N. S. Kim, "Approximate computing: A survey," *IEEE Des. Test*, vol. 33, no. 1, pp. 8–22, Feb. 2016.
- [3] D. Tian et al., "A low-latency power series approximate computing and architecture for co-calculation of division and square root," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 6, pp. 2723–2734, Jun. 2024, doi: 10.1109/TCSI.2024.3368102.
- [4] M. H. Haider, H. Zhang, and S. B. Ko, "Decoder reduction approximation scheme for booth multipliers," *IEEE Trans. Comput.*, vol. 73, no. 3, pp. 735–746, Mar. 2024, doi: 10.1109/TC.2023.3343093.
- [5] A. Towhidi, R. Omidi, and K. Mohammadi, "On the design of iterative approximate floating-point multipliers," *IEEE Trans. Comput.*, vol. 72, no. 6, pp. 1623–1635, Jun. 2023, doi: 10.1109/TC.2022.3216465.
- [6] C. Yan, X. Zhao, T. Zhang, J. Ge, C. Wang, and W. Liu, "Design of high hardware efficiency approximate floating-point FFT processor," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 11, pp. 4283–4294, Nov. 2023, doi: 10.1109/TCSI.2023.3298882.
- [7] K. Chen, C. Xu, H. Waris, W. Liu, P. Montuschi, and F. Lombardi, "Exact and approximate squarers for error-tolerant applications," *IEEE Trans. Comput.*, vol. 72, no. 7, pp. 2120–2126, Jul. 2023, doi: 10.1109/TC.2022.3228592.
- [8] C. Chen, W. Qian, M. Imani, X. Yin, and C. Zhuo, "PAM: A piecewise-linearly-approximated floating-point multiplier with unbiasedness and configurability," *IEEE Trans. Comput.*, vol. 71, no. 10, pp. 2473–2486, Oct. 2022, doi: 10.1109/TC.2021.3131850.
- [9] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proc. IEEE*, vol. 108, pp. 2108–2135, Dec. 2020.
- [10] H. A. F. Almurib, T. N. Kumar, and F. Lombardi, "Inexact designs for approximate low power addition by cell replacement," in *Proc. Des., Automat. Test Eur. Conf. Exhib. (DATE)*, 2016, pp. 660–665.
- [11] S. Raj, U. Chaurasia, A. Bahukhandi, and P. Mittal, "Hybrid approximate adders using pass transistor logic and transmission gate," in *Advanced Computational Paradigms and Hybrid Intelligent Computing*, Singapore: Springer, 2022, pp. 273–284.
- [12] C. K. Jha and J. Mekie, "SEDA - Single exact dual approximate adders for approximate processors," in *Proc. 56th ACM/IEEE Des. Automat. Conf. (DAC)*, 2019, pp. 1–2.
- [13] H. Waris, C. Wang, and W. Liu, "High-performance approximate half and full adder cells using NAND logic gate," *IEICE Electron. Exp.*, vol. 16, no. 6, pp. 20190043–20190043, 2019.
- [14] S. Dutt, S. Nandi, and G. Trivedi, "Analysis and design of adders for approximate computing," *ACM Trans. Embedded Comput. Syst.*, vol. 17, no. 2, pp. 1–28, 2018.
- [15] H. Cai, Y. Wang, L. A. B. Naviner, W. Zhao, and W. Zhao, "Approximate computing in MOS/spintronic non-volatile full-adder," in *Proc. IEEE/ACM Int. Symp. Nanoscale Archit. (NANOARCH)*, 2016, pp. 203–208.
- [16] S. Angizi, H. Jiang, R. F. DeMara, J. Han, and D. Fan, "Majority-based spin-CMOS primitives for approximate computing," *IEEE Trans. Nanotechnol.*, vol. 17, no. 4, pp. 795–806, Jul. 2018.
- [17] A. K. Verma, P. Brisk, and P. Ienne, "Variable latency speculative addition: A new paradigm for arithmetic circuit design," in *Proc. Des., Automat. Test Eur.*, 2008, pp. 1250–1255.
- [18] D. Esposito, D. D. Caro, and A. G. M. Strollo, "Variable latency speculative parallel prefix adders for unsigned and signed operands," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 63, no. 8, pp. 1200–1209, Aug. 2016.
- [19] H. Ghabeli, A. Sabbagh Molahosseini, A. A. Emrani Zarandi, and L. Sousa, "Variable latency carry speculative adders with input-based dynamic configuration," *Comput. Elect. Eng.*, vol. 93, 2021, Art. no. 107247.
- [20] B. Garg and S. K. Patel, "Reconfigurable carry look-ahead adder trading accuracy for energy efficiency," *J. Signal Process. Syst.*, vol. 93, no. 1, pp. 99–111, 2020.
- [21] S. K. Patel, B. Garg, and S. K. Rai, "An efficient accuracy reconfigurable CLA adder designs using complementary logic," *J. Electron. Test.*, vol. 36, no. 1, pp. 135–142, 2020.
- [22] Z. Ning, W. L. Goh, and K. S. Yeo, "An enhanced low-power high-speed adder for error-tolerant application," in *Proc. 12th Int. Symp. Integr. Circuits*, 2009, pp. 69–72.
- [23] M. Shafique, W. Ahmad, R. Hafiz, and J. Henkel, "A low latency generic accuracy configurable adder," in *Proc. 52nd ACM/EDAC/IEEE Des. Automat. Conf. (DAC)*, 2015, pp. 1–6.
- [24] M. A. Hanif, R. Hafiz, O. Hasan, and M. Shafique, "QuAd: Design and analysis of quality-area optimal low-latency approximate adders," in *Proc. 54th ACM/EDAC/IEEE Des. Automat. Conf. (DAC)*, 2017, pp. 1–6.

- [25] S. Dutt, S. Nandi, and G. Trivedi, "Accuracy enhancement of equal segment based approximate adders," *IET Comput. Digit. Techn.*, vol. 12, no. 5, pp. 206–215, 2018.
- [26] F. Frustaci, S. Perri, P. Corsonello, and M. Alioto, "Energy-quality scalable adders based on non-zeroing bit truncation," *IEEE Trans. Very Large Scale Integr.*, vol. 27, no. 4, pp. 964–968, Apr. 2019.
- [27] O. J. Bedrij, "Carry-select adder," *IRE Trans. Electron. Comput.*, vol. EC-11, no. 3, pp. 340–346, 1962.
- [28] K. Du, P. Varman, and K. Mohanram, "High performance reliable variable latency carry select addition," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, 2012, pp. 1257–1262.
- [29] I. Lin, Y. Yang, and C. Lin, "High-performance low-power carry speculative addition with variable latency," *IEEE Trans. Very Large Scale Integr.*, vol. 23, no. 9, pp. 1591–1603, Sep. 2015.
- [30] J. Hu and W. Qian, "A new approximate adder with low relative error and correct sign calculation," in *Proc. Des., Automat. Test Eur. Conf. Exhib. (DATE)*, 2015, pp. 1449–1454.
- [31] W. Xu, S. S. Sapatnekar, and J. Hu, "A simple yet efficient accuracy-configurable adder design," *IEEE Trans. Very Large Scale Integr.*, vol. 26, no. 6, pp. 1112–1125, Jun. 2018.
- [32] F. Ebrahimi-Azandaryani, O. Akbari, M. Kamal, A. Afzali-Kusha, and M. Pedram, "Block-based carry speculative approximate adder for energy-efficient applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 1, pp. 137–141, Jan. 2020.



Tayebah Karimi received the B.Sc. degree in computer engineering from Kermanshah University of Technology. She is currently working toward the master's degree in computer architecture with Razi University, Kermanshah, Iran. Her research interests include digital circuit design and approximate computing.



Arezoo Kamran received the B.Sc. degree in computer engineering from the University of Tehran, the M.Sc. degree in computer architecture from Sharif University of Technology, and the Ph.D. degree in computer architecture from the University of Tehran. She then joined the Department of Computer Engineering, Razi University in Kermanshah, Iran, in 2015. Her research interests include approximate computing, hardware for AI and machine learning, high-performance computing, development of EDA tools, digital circuit design and testing, and the reliability of digital circuits.

ability of digital circuits.